

MTE 423

Automatic Control 2

Course Project

Submitted by

Tarek Mahmoud Shohdy 120200030

Mostafa Ahmed Mostafa 120200214

To

Prof. Ayman Nada

I. Introduction

A self-balancing robot is a type of robot that maintains balance on two wheels. This report outlines the development process of our self-balancing robot, which includes an Arduino board, an MPU-6050 sensor, and DC motors. The project aims to design and implement two control algorithms for the robot, compare their efficiency, and test its performance under various conditions. The robot is constructed with a two-wheeled structure and a platform for carrying payloads. The MPU-6050 sensor measures the robot's tilt angle, and the Arduino board controls the motors based on the implemented control algorithm.

I.1 Hardware Overview



1- Motors:
Two NEMA MiniQ motor wheel sets with a speed of 120 rpm at 6V are employed for locomotion. These motors provide precise control and are integral to achieving the desired movement and balance of the self-balancing robot.



2- Sensors:
The MPU-6050, featuring an accelerometer and gyroscope, serves as the primary sensor. This component is essential for real-time measurement of the robot's tilt angle, enabling accurate feedback for the control algorithms.



3-Power Supply:
The robot is powered by two 18650 Li-ion batteries, each providing 3.7V and 2400mAh capacity. This power supply configuration ensures a reliable and sustained source of energy for the entire system.



4-Motor Driver:
A Dual H-bridge DC stepper motor controller L298N module is utilized to manage and control the DC motors. This motor driver module facilitates the precise regulation of motor speed and direction, contributing to the robot's stability and movement control.



5-Controller:
The Arduino UNO serves as the central control unit. With its versatility and compatibility with various sensors, the Arduino UNO processes data from the MPU-6050 and executes the control algorithms. It plays a crucial role in coordinating motor responses to maintain the robot's balance.

6- Chassis:
The robot's structural framework is a 3D printed design with dimensions of 40x50x80 mm. This carefully crafted chassis provides the necessary support for the hardware components, ensuring a balanced and stable platform for the self-balancing robot.



II. System modeling

The state-space model was developed based on the book "Development of a Self-Balancing Two-Wheeled Robot", using Lagrange equation referenced to figure 1.
And figure 2 represents the robot axes. More details about the dynamic model is provided:

Total Kinetic Energy

$$x_p = x + lsin(\theta_p) \quad (1)$$

$$z_p = lcos(\theta_p) \quad (2)$$

$$KE_{trans-C} = \frac{1}{2}m[\dot{x}_p^2 + \dot{z}_p^2] \quad (3)$$

$$KE_{trans-C} = \frac{1}{2}m[(v + lw_pcos(\theta_p))^2 + (-lw_psin(\theta_p))^2] \quad (4)$$

$$KE_{trans-C} = \frac{1}{2}m(v^2 + l^2w_p^2) + mvlw_pcos(\theta_p) \quad (5)$$

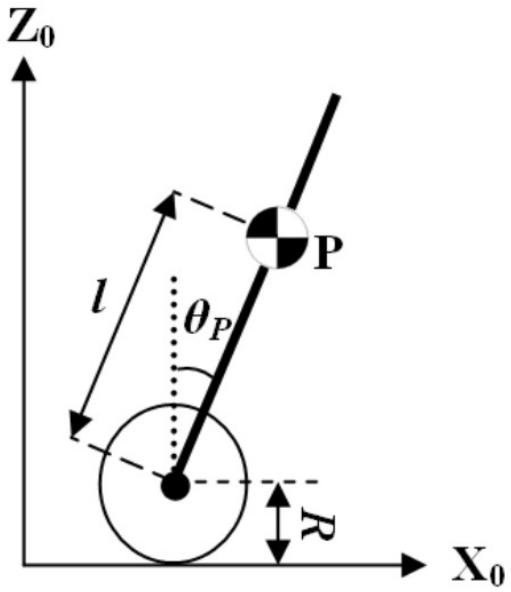


Figure 1 Self-Balancing Robot parameters

$$KE_{rot-C} = \frac{1}{2}J_cw_p^2 \quad (6)$$

$$KE_{total-C} = KE_{trans-C} + KE_{rot-C} \quad (7)$$

$$KE_{total-C} = \frac{1}{2}m(v^2+l^2w_p^2)+mvlw_pcos(\theta_p)+\frac{1}{2}J_cw_p^2 \quad (8)$$

$$KE_{trans-W} = Mv^2 \quad (9)$$

$$KE_{rot-W} = J_w\frac{v^2}{R^2} \quad (10)$$

$$KE_{total-W} = KE_{trans-W} + KE_{rot-W} \tag{11}$$

$$KE_{total-W} = Mv^2 + J_w \frac{v^2}{R^2} \tag{12}$$

$$KE_{total} = KE_{total-C} + KE_{total-W} \tag{13}$$

$$KE_{total} = \frac{1}{2}mv^2 + \frac{1}{2}ml^2w_p^2 + mvlw_p\cos(\theta_p) + \frac{1}{2}J_cw_p^2 + Mv^2 + J_w\frac{v^2}{R^2} \tag{14}$$

Total Potential Energy

$$PE_{total} = mglcos(\theta_P) \tag{15}$$

Dissipation Energy

$$F = \mu_o v^2 + \mu_1 w_p^2 \tag{16}$$

Where μ_o is the coefficient of friction between the wheels and the terrain, and μ_1 is the coefficient of friction between the chassis and the wheels rotational axis.

Lagrange Equation

$$\frac{\partial(\frac{\partial KE}{\partial \dot{q}_i}))}{\partial t} - \frac{\partial KE}{\partial q_i} + \frac{\partial F}{\partial \dot{q}_i} + \frac{\partial PE}{\partial q_i} = Q_i \tag{17}$$

Where the q_i is a generalized state variable, t is the time, and Q_i is the external forces which is the torque exerted by the motors on the wheels.

From substituting in the Lagrange equation (Equ 17), we get the two differential equations governing the dynamics of the system, Where $\tau_L + \tau_R$ are the left and right wheel torques respectively.

$$(m + 2M + \frac{2J_w}{R^2})\ddot{x} + mlcos(\theta_p)\ddot{\theta_p} + 2\mu_o v = 0 \tag{18}$$

$$mlcos(\theta_p)\ddot{x} + (ml^2 + J_c)\ddot{\theta_p} + 2\mu_1 w_p + (mvw_p - mg)lsin(\theta_p) = \tau_L + \tau_R \tag{19}$$

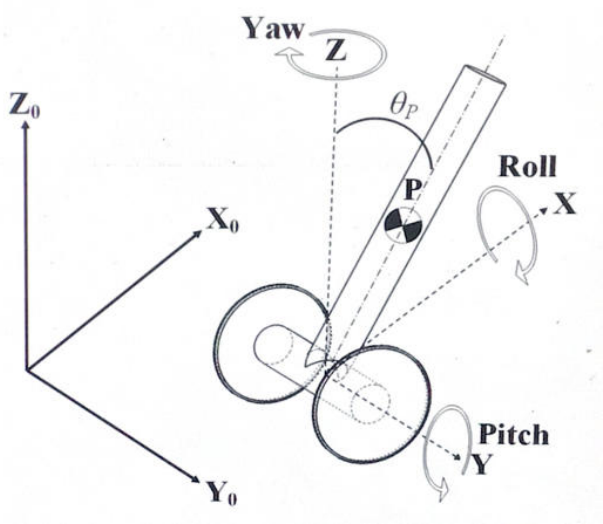


Figure 2 Self-Balancing Robot parameters

$\because \theta_p$ is considered to always be less than $10°$
$\therefore$ The following assumptions are valid:

(1) $\sin(\theta_p) = \theta_p$
(2) $\cos(\theta_p) = 1$

$$(m + 2M + 2\frac{J_w}{R^2})\ddot{x} + ml\ddot{\theta_p} + 2\mu_o v = 0 \qquad (20)$$

$$ml\ddot{x} + (ml^2 + J_c)\ddot{\theta_p} + (mvlw_p - mgl)\theta_p + 2\mu_1 w_p = \tau_L + \tau_R \qquad (21)$$

Calculating the state space matrices (A, B, C, D) based on the model derived in the book and our robot parameters. See figure 3 and figure 4.

$$\dot{x} = Ax + Bu \qquad (22)$$

$$x = \begin{bmatrix} x \\ \theta_p \\ \dot{x} \\ \dot{\theta}_p \end{bmatrix} = \begin{bmatrix} x \\ \theta_p \\ v \\ w_p \end{bmatrix} \qquad (23)$$

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{(ml)^2 g}{\text{den}} & -\frac{2\mu_o(ml^2 + J_c)}{\text{den}} & \frac{2ml\mu_1}{\text{den}} \\ 0 & \frac{nom}{\text{den}} & \frac{2\mu_0 ml}{\text{den}} & -\frac{2\mu_1 * nom}{\text{den}} \end{bmatrix} \qquad (24)$$

$$B = \begin{bmatrix} 0 \\ 0 \\ -\frac{ml}{den} \\ \frac{nom}{den} \end{bmatrix} \qquad (25)$$

$$C = [1\ 0\ 0\ 0] \qquad (26)$$

$$D = [0] \qquad (27)$$

$$den = (m + 2M + 2\frac{J_w}{R^2})(ml^2 + J_c) - (ml)^2 \qquad (28)$$

$$nom = m + 2M + 2\frac{J_w}{R^2} \qquad (29)$$

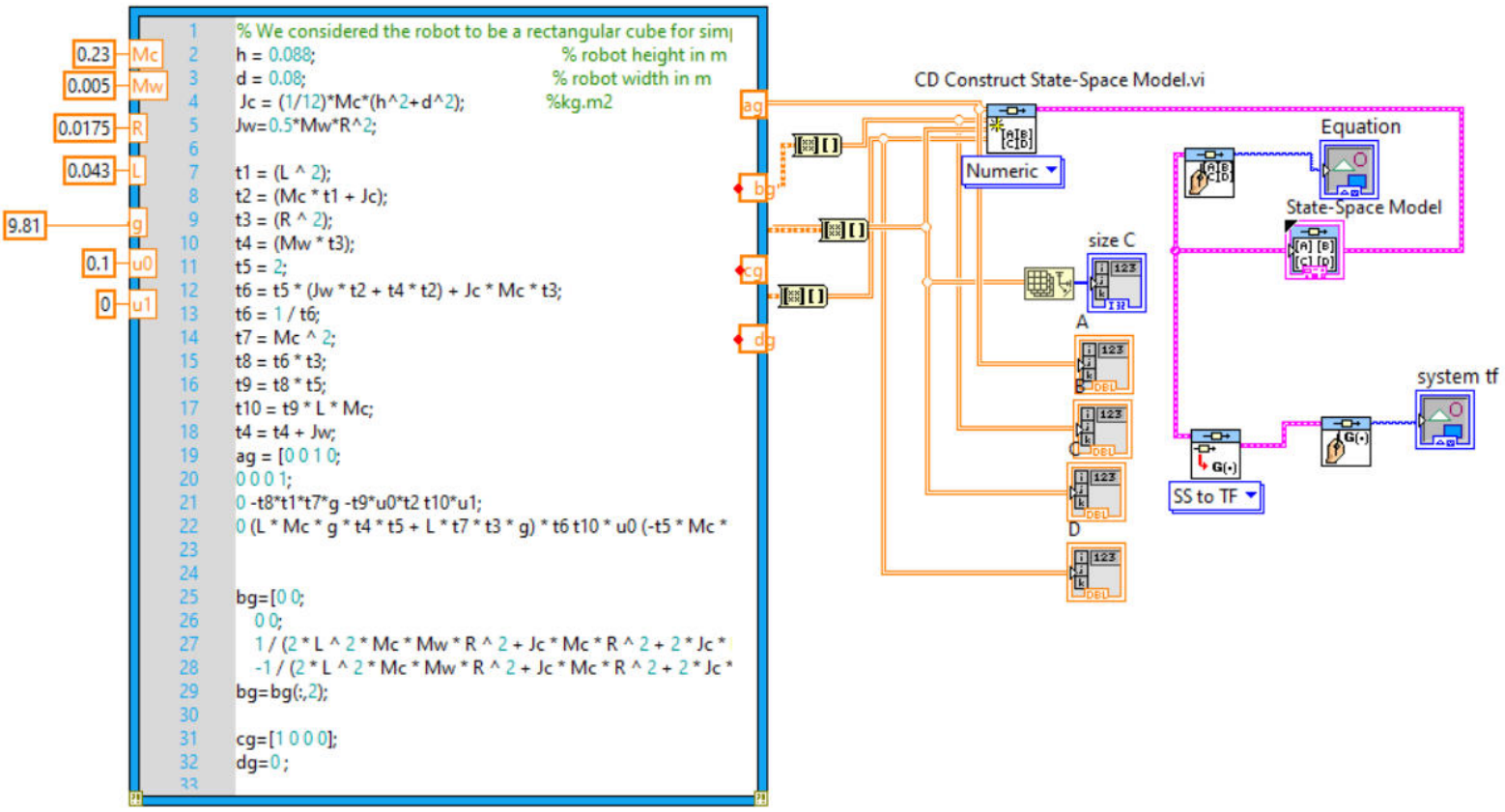


Figure 3 Self-Balancing Robot State Space Script

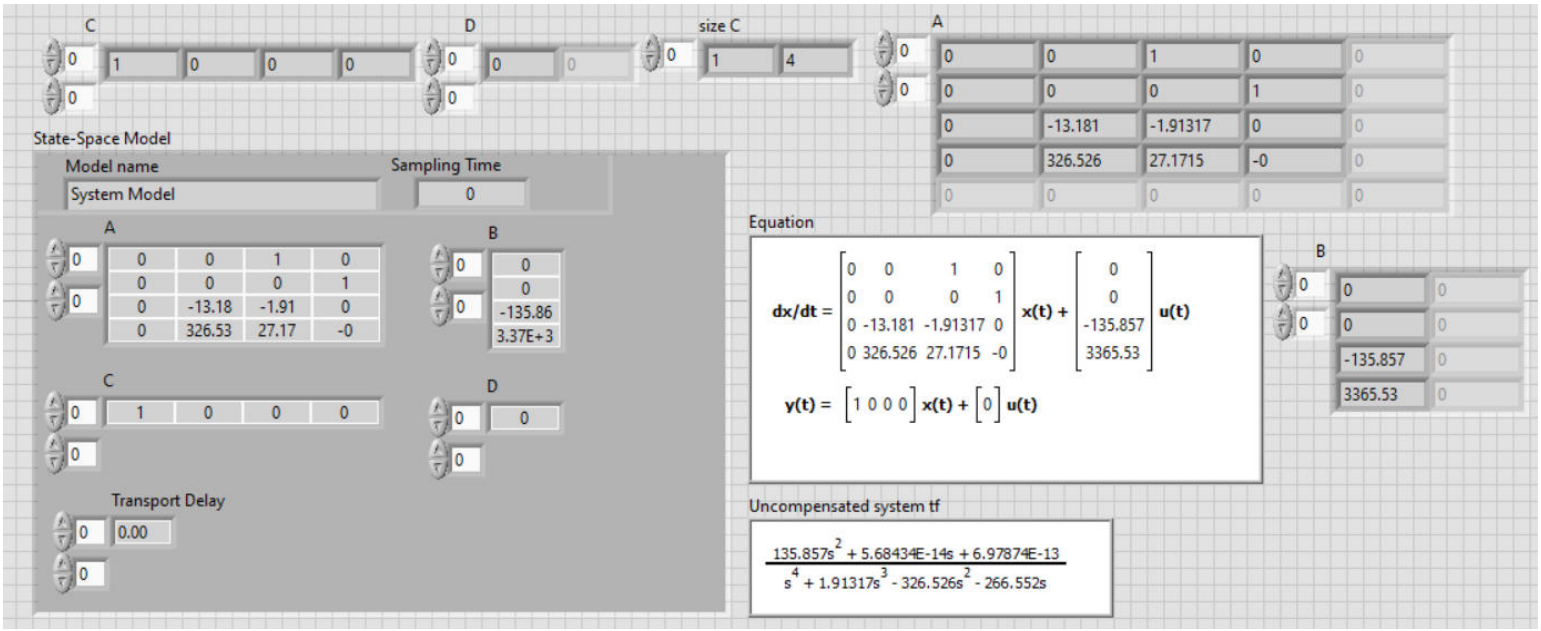


Figure 4 Self-Balancing Robot State Space & Transfer Function

III. Control Design

A VI on LabVIEW was designed to convert the state space model into a transfer function with an input "tilting angle" and output "motor PWM". So, in the following figures 5,6, there are 3 different root loci.

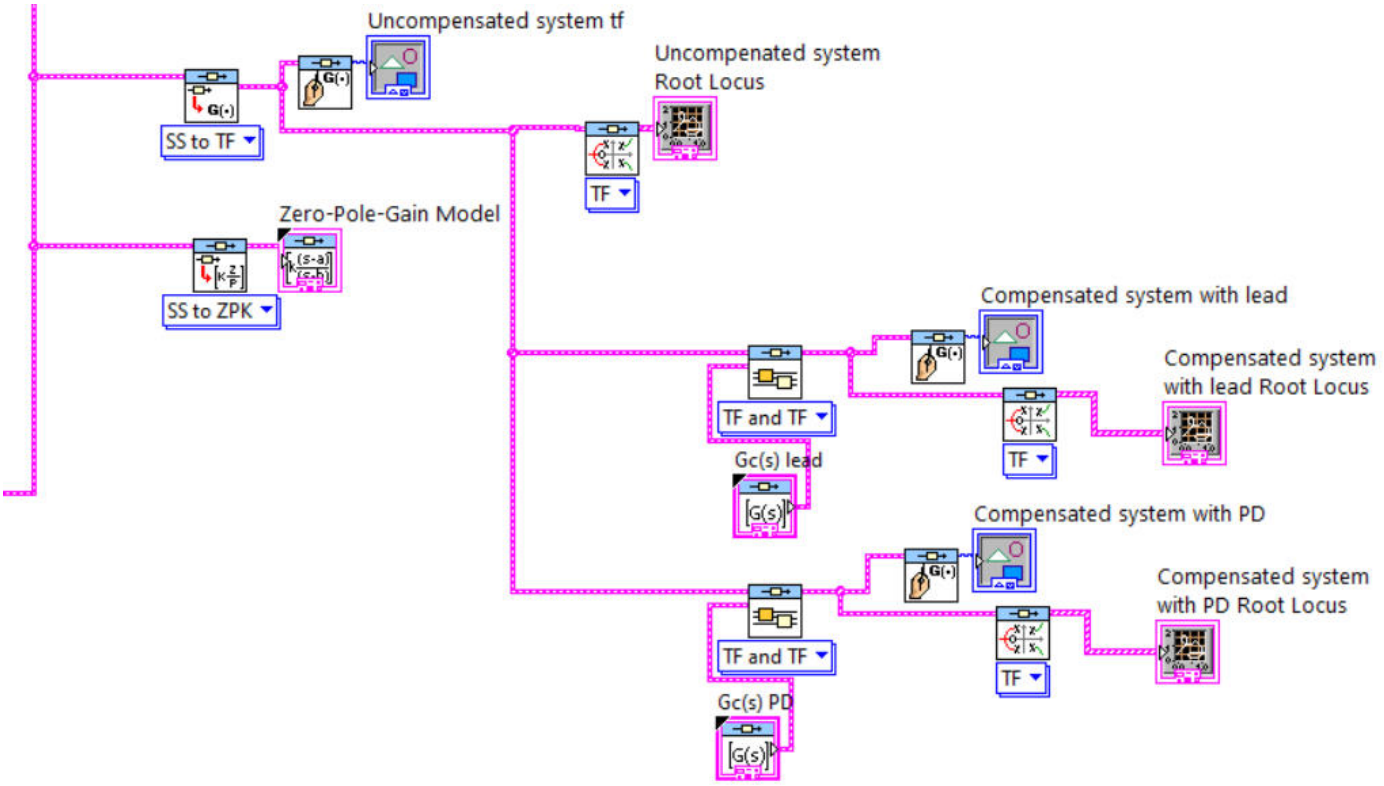


Figure 5*: Back* Panel of Uncompensated and Compensated systems

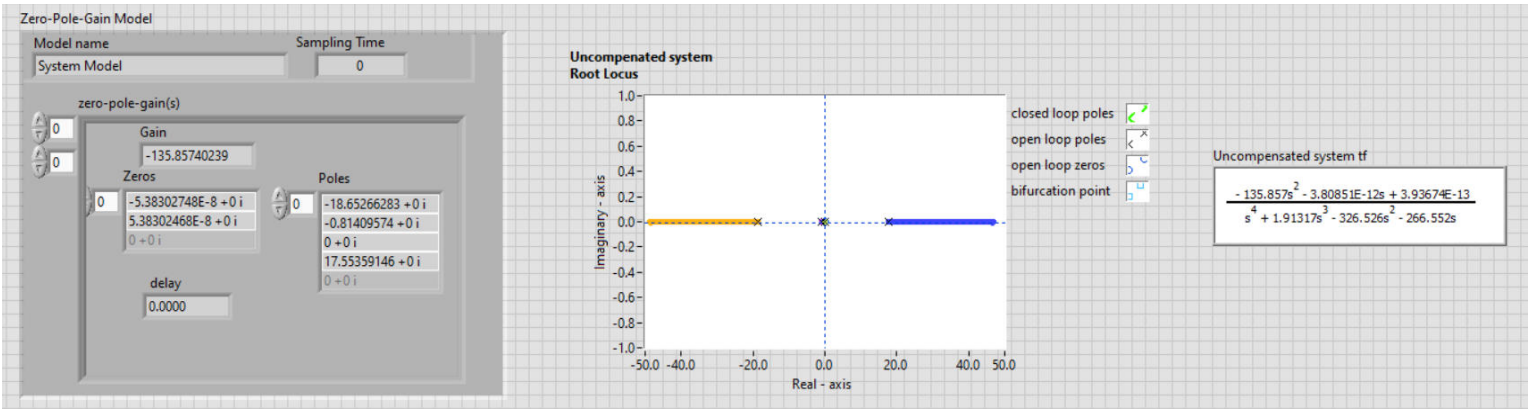


Figure 6*: Front* Panel of Uncompensated

Realizing that the root locus is not similar to the normal self-balancing robots' root loci, so we compared the the transfer function and realized that we had a missing gain with -1 as shown in figures 7 and 8.

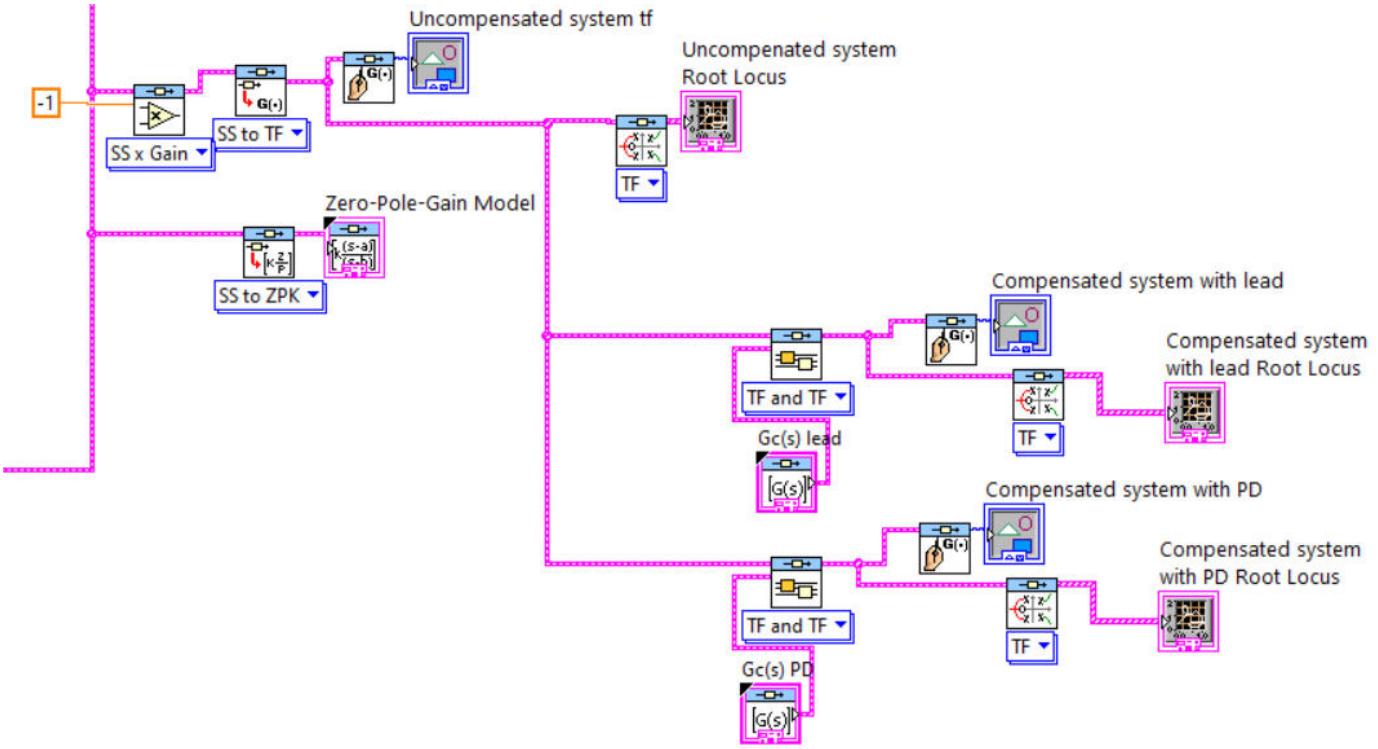


Figure 7: Back Panel of Uncompensated and Compensated systems *with multiplying by -1*

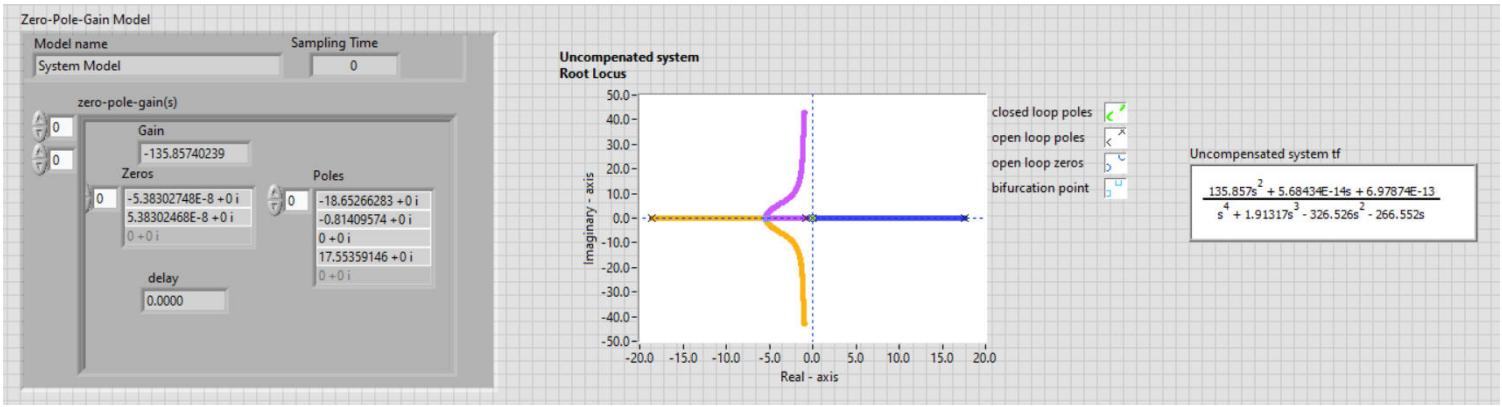


Figure 8: Front Panel of Uncompensated *with multiplying by -1*

In order to stabilize in a better way, we designed a PD controller with fine-tuning the its parameters as shown in the following figure 9 representing the PD compensator and also figure 10 after the -1 modification.

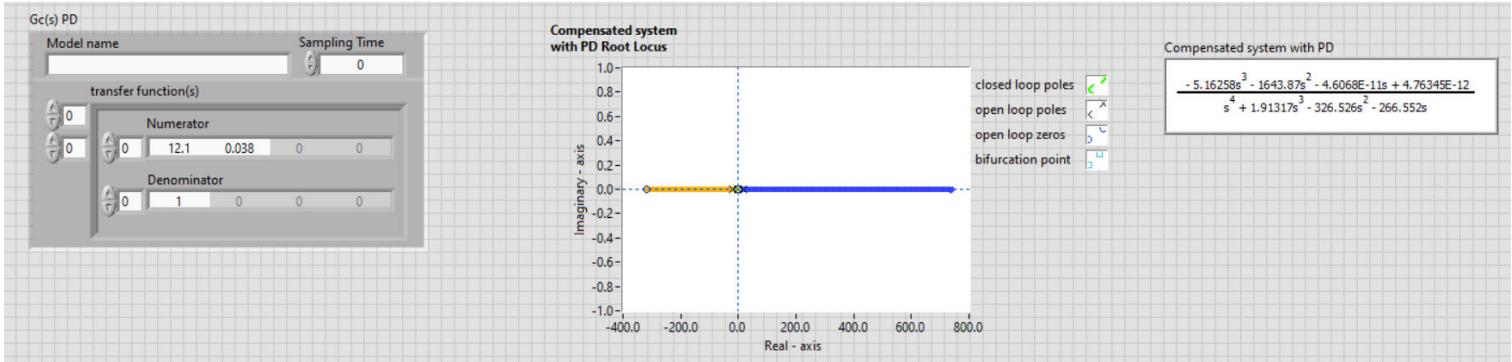


Figure 9: Front Panel of Compensated system with PD with Kp = 12.1 and Kd = 0.038

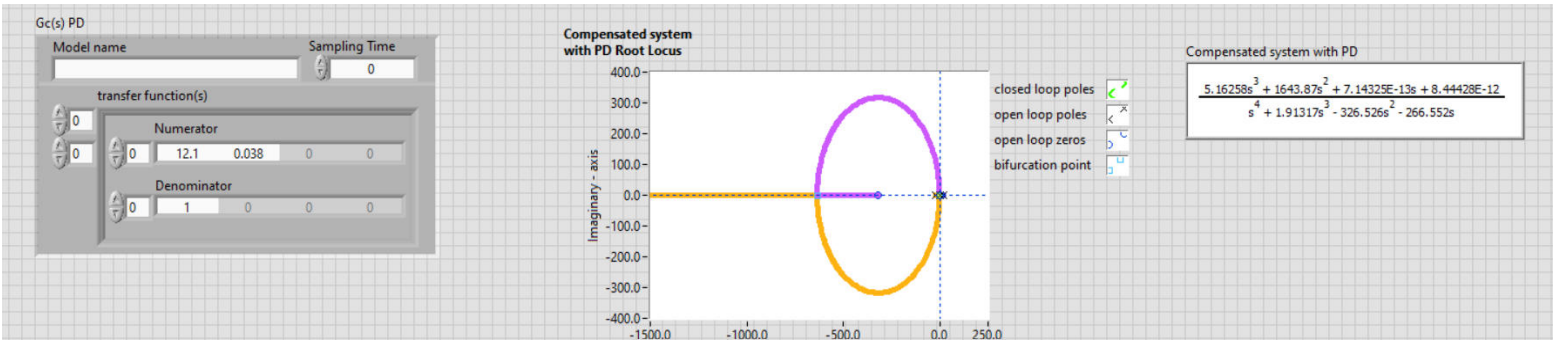
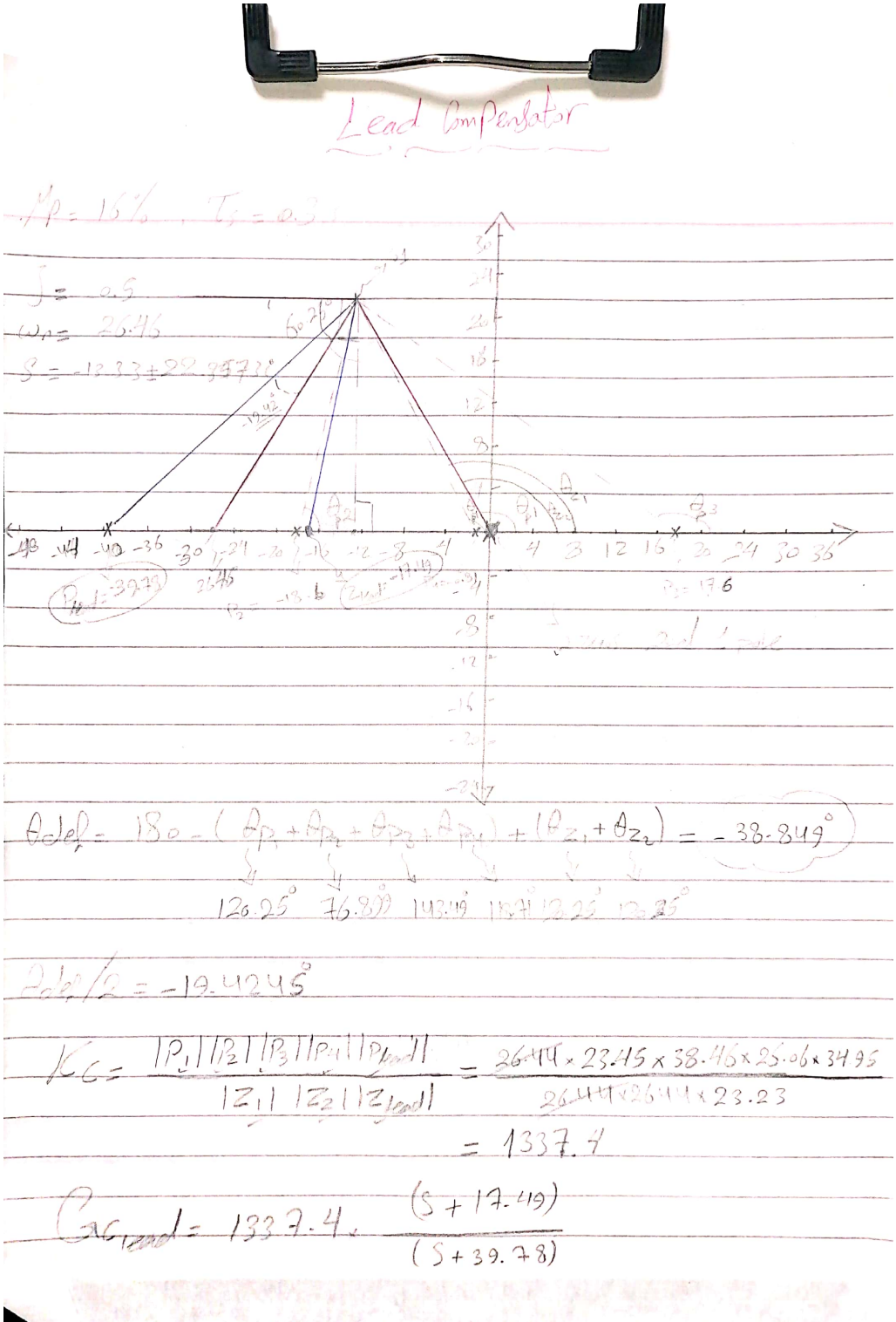


Figure 10*: Front* Panel of Compensated system with PD
with Kp = 12.1 and Kd = 0.038
and with multiplying by -1

In order to compare the performance with another compensator, we designed a Lead compensator by determining the needed overshoot percentage and desired settling time as shown in the calculation below and the root locus in figures 11 and 12.

The calculations of this lead compensator are done as shown:



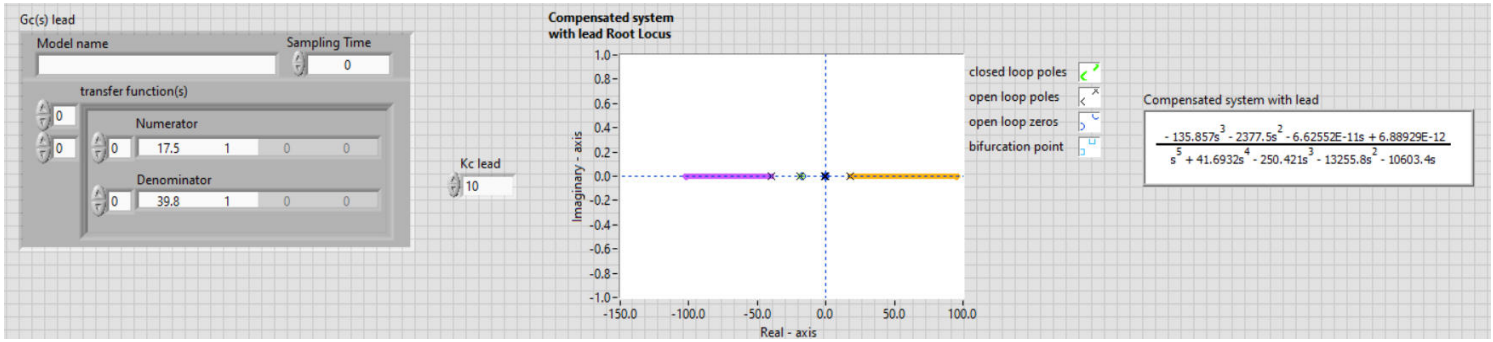


Figure 11: *Front* Panel of Compensated system with lead

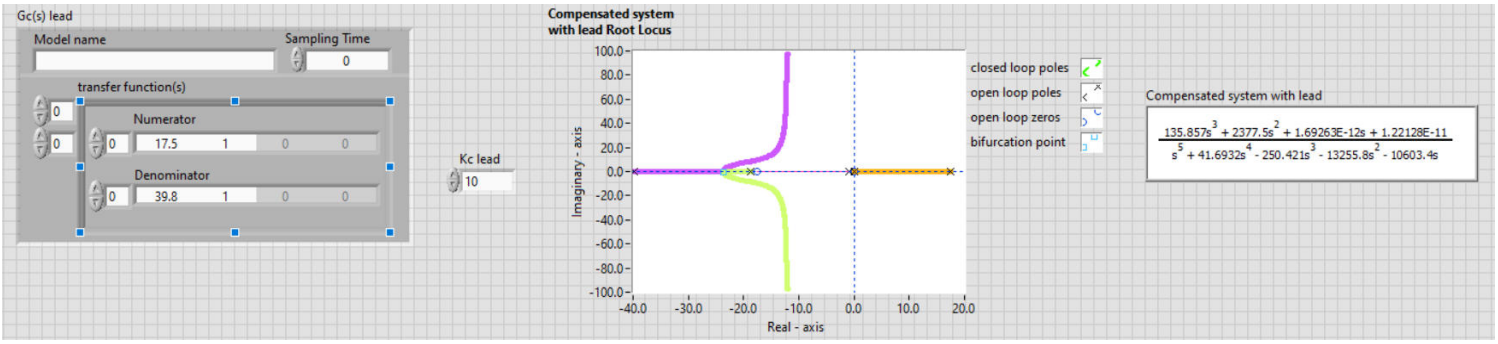


Figure 12: *Front* Panel of Compensated system with lead *with multiplying by -1*

IV. Implementation

The LabVIEW LINX Toolkit was used to interface with the Arduino UNO using serial communication. The interface was designed to read both the accelerometer and gyroscope readings from the MPU6050. The tilt angle was estimated using a complementary filter where both readings were taken into consideration. In figur es 13 and 14 are both the front panel and block diagrams of the LINX VI for control implementation. In figures 15 and 16 we discuss the LINX interface SubVIs in detail.

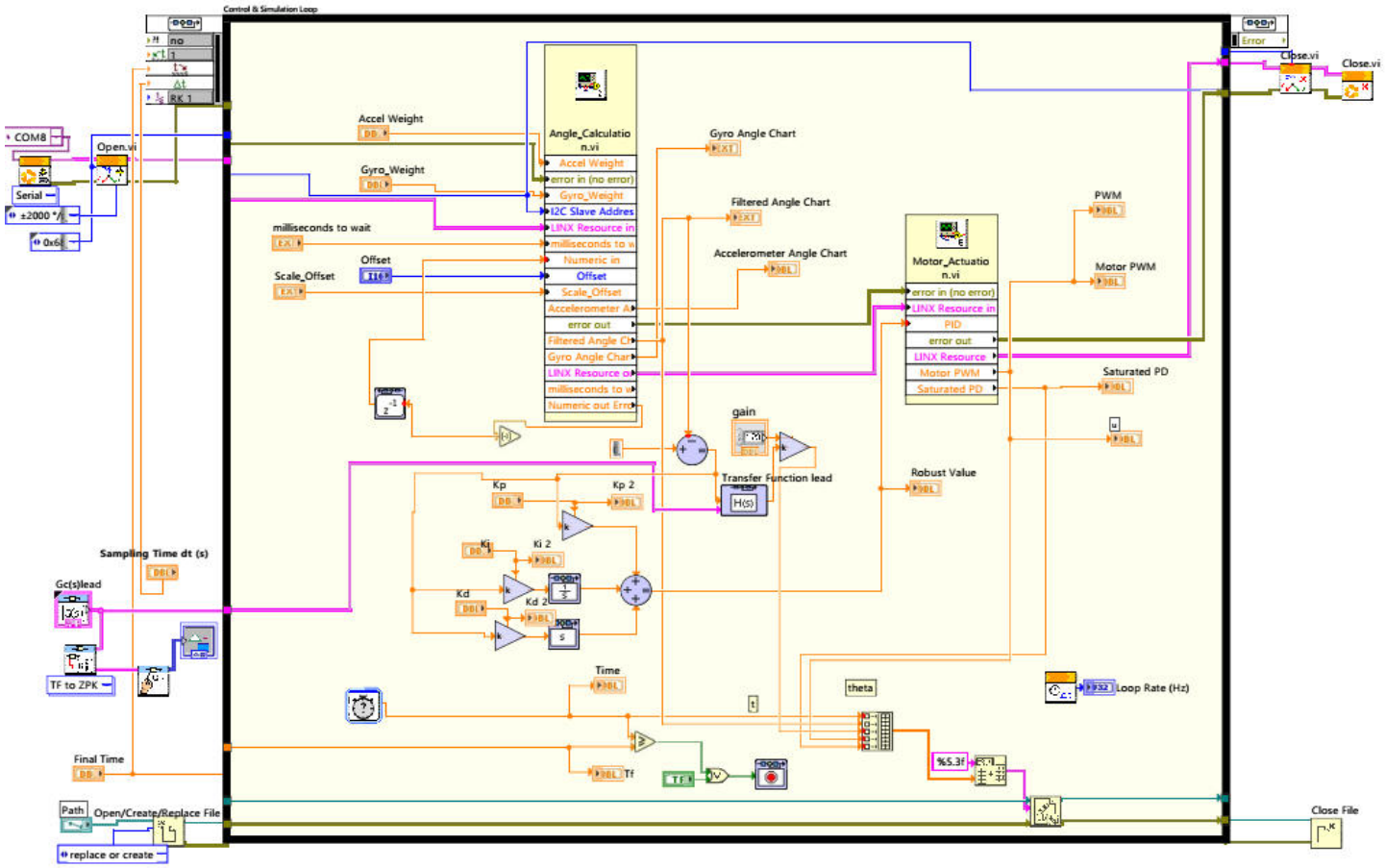


Figure 13: Back Panel of LINX interface code

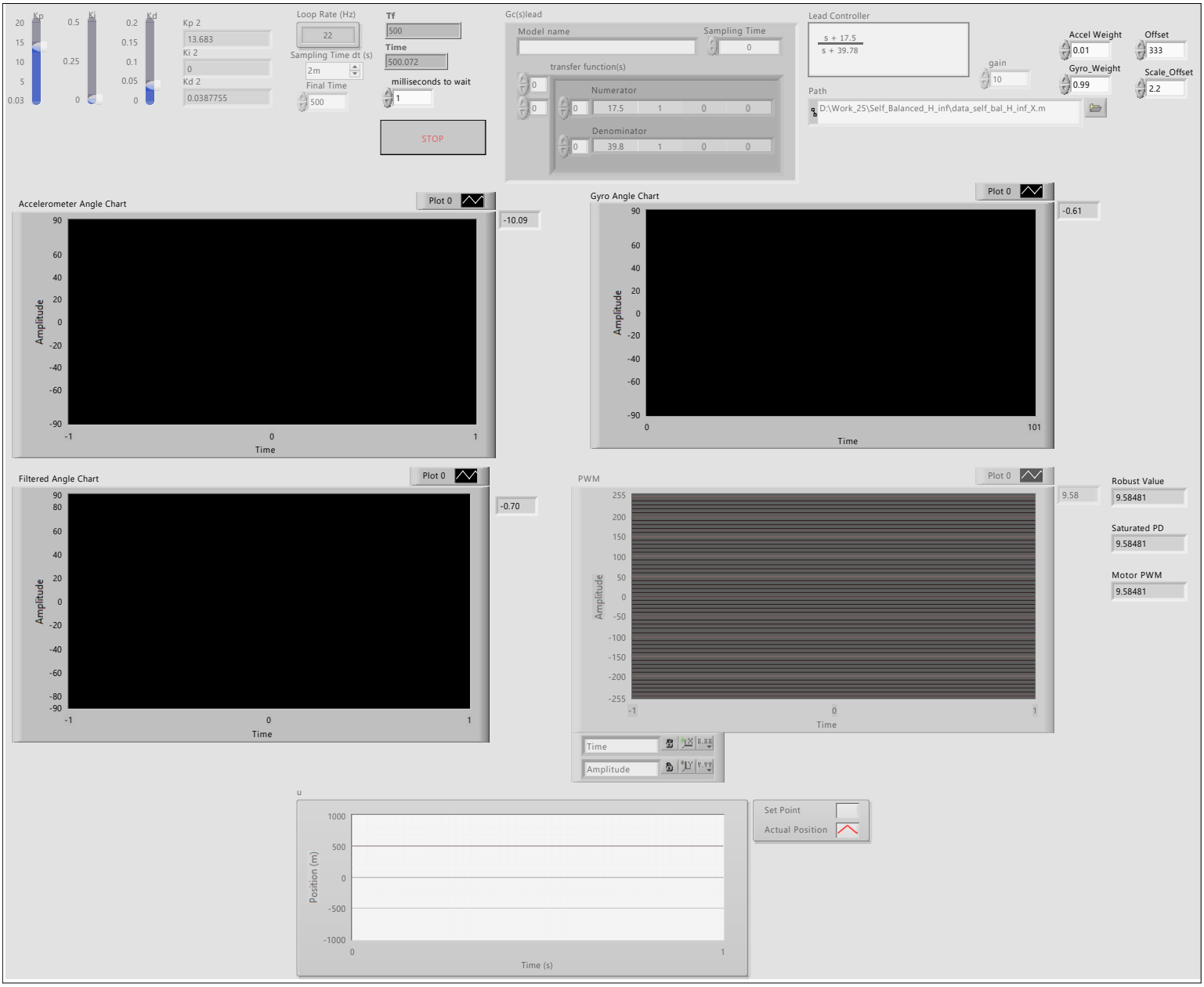


Figure 14: Back Panel of LINX interface code

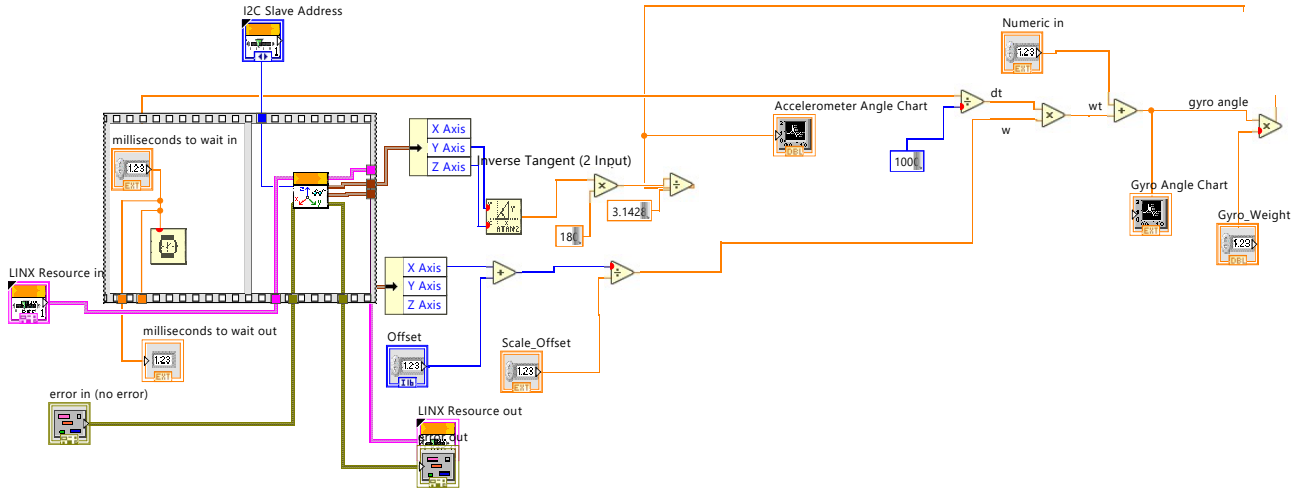


Figure 15: Angle_Calculation SubVI

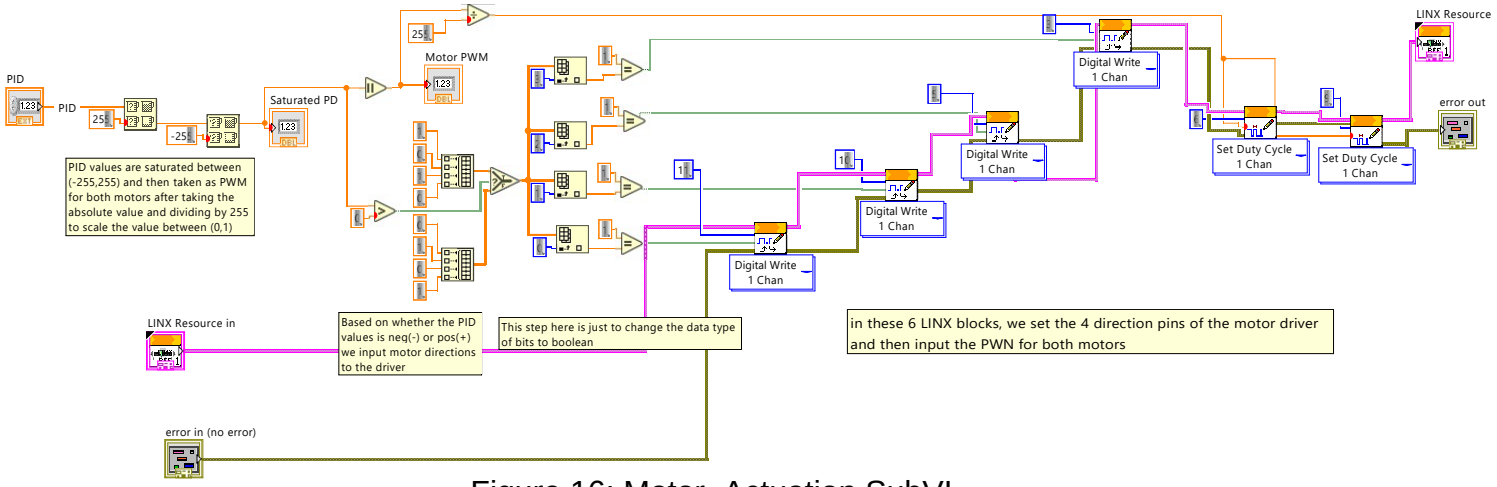


Figure 16: Motor_Actuation SubVI

V. Calibration and Fine-tuning

The MPU-6050 sensor was calibrated to correct for scale offset and shift offset errors. The calibration process was done by measuring the angle in multiple positions to get the offset values of the sensor. In our case, we were tried using a complementary filter which normally solves the problems of gyroscope and accelerometer inaccuracies. However, in our case, the accelerometer readings were so noisy, so they caused the robot to overshoot a lot which causes system failure. Due to this, The most reasonable response we got was using the gyroscope readings only, but this leads us back to the gyroscope problem. gyroscopes are subject to bias instabilities, in which the initial zero reading of the gyroscope will cause drift over time due to integration of inherent imperfections and noise within the device. This means that it is very dependent on the initial condition. Once the sensor was calibrated, the PID controller was fine-tuned to achieve good performance for the robot. During the fine-tuning, the robot was tested under different surfaces and so we noticed that friction has a very large impact on the system response. In some cases, we would even need to tune the PID parameters all over again. This is somehow expected since we assumed a coefficient of friction of 0.1 when modeling. So our system is not robust when it comes to changing friction.

VI. Results and Analysis

1-Balancing Behavior with Onboard Batteries:

Initially, attempts were made to balance the self-balancing robot with the batteries fixed directly onto its surface. Surprisingly, the robot failed to maintain stability under this configuration. The added weight and the resulting change in the center of gravity seemed to adversely affect the robot's balancing capabilities. The increased inertia introduced by the fixed batteries likely contributed to the instability.

2-Improved Balancing Performance with Wired Configuration:

A significant improvement in balancing performance was observed when the batteries were detached from the robot chassis and connected to it via wires from a distance. This alteration allowed for more flexibility in the weight distribution and alleviated the adverse effects on the robot's balance.

3-PD Control Parameters for Successful Balancing:

PD Control (Kp = 12.1, Kd = 0.038)

Implementation of Proportional-Derivative (PD) control played a crucial role in achieving stable balance. By fine-tuning the PD control parameters, specifically setting Kp=12.1 and Kd=0.038, the robot demonstrated successful self-balancing behavior. The proportional term (Kp) contributed to the correction of the current error, while the derivative term (Kd) helped dampen oscillations, resulting in a smoother and more stable response.

4-Comparison with Lead Compensator:
An alternative control strategy, the Lead Compensator, was also explored. A lead compensator with settling time set to 300 ms and overshoot limited to 16% was able to achieve stability. However, comparative analysis revealed that the PD control approach exhibited higher efficiency in maintaining balance.

5-Efficiency Comparison: PD Control vs. Lead Compensator:
The efficiency of the balancing system was assessed through experimentation with both PD control and Lead Compensator. While the Lead Compensator showed satisfactory performance, the PD control mechanism outperformed it in terms of responsiveness, less overshoot and overall stability. The robot exhibited improved tracking of the desired pitch angle, responded promptly to disturbances, and showcased better disturbance rejection capabilities.

VII. Conclusion

The results suggest that the effectiveness of the self-balancing robot is highly dependent on the distribution of weight and the tuning of control parameters. The wired configuration, along with the optimized PD control parameters, emerged as the more successful combination for achieving stable and reliable self-balancing behavior. Further investigation into the impact of varying weights and control parameter tuning may provide additional insights for enhancing the robot's performance in diverse scenarios.

VIII. Future Work

1. Tracking problem
One significant avenue for future work involves extending the capabilities of the self-balancing robot beyond mere stabilization. The implementation of trajectory tracking algorithms will allow the robot to follow predefined paths or respond to dynamic commands, demonstrating a more versatile and practical use in real-world scenarios. Investigate control strategies and algorithms tailored for trajectory tracking while ensuring robust stability.

2. Sensor Fusion Techniques
To enhance the accuracy and robustness of the sensor measurements, consider implementing advanced sensor fusion techniques. Particle filters or Kalman filters can be employed to effectively fuse data from multiple sensors, reducing noise and errors in the measurements. This will contribute to more precise state estimation, thereby improving the overall performance of the self-balancing system.

3. Robust Control Strategies (H-infinity Control)
Explore the application of robust control strategies, particularly H-infinity control techniques. Robust control methods are designed to handle uncertainties and disturbances in the system. Implementing H-infinity control can potentially improve the self-balancing system's ability to handle external disturbances and parameter variations, leading to increased robustness in real-world scenarios.

4. Hardware Component Upgrades
Upgrade the hardware components of the self-balancing robot to enhance its overall efficiency and capabilities. Consider replacing the current 18650 Li-ion batteries with smaller, lightweight alternatives that maintain or increase the power capacity. Additionally, replacing the Arduino Uno with an ESP32 microcontroller offers advantages such as a smaller form factor, built-in Wi-Fi capabilities, and increased computational power. The integration of Wi-Fi communication facilitates remote control and real-time adjustments, making it particularly useful for trajectory tracking applications.

5. Wireless communication between LabVIEW and Arduino

Building the code on the Arduino Compatible Compiler for LabVIEW instead of using a firmware library such as LINX, allows for the binary code to be uploaded to the on-board flash memory of the target Arduino board. This would enable the microcontroller on the board to execute the code, in a standalone fashion, without the need for a constant communication link with a host computer once the code is uploaded. This would make the embedded system more autonomous, and thus more suited for field deployment in portable and practical applications, as it eliminates the need for a PC.

IX. References

1. Abdelgawad, A. A., Shohdy, T. M., & Nada, A. A. (2024a, February). Robust H-infinity Control of a Two-Wheeled Self-Balancing Robot.
2. "Development of a Self-Balancing Two-Wheeled Robot" by X. Chen, Z. Lu, and Y. Liu. This book was used as a reference to develop the state-space model and to choose the appropriate control method for the robot.
3. "MPU-6050 6-Axis Accelerometer/Gyroscope Datasheet" by InvenSense. This datasheet provided detailed information about the MPU-6050 sensor, including its specifications and the calibration process.
4. "Arduino Compatible Compiler for LabVIEW" by Aledyne-TSXperts. This tool was used to upload the LabVIEW code to the Arduino board.
5. Wang, Y., Mao, W., Wang, Q., and Xin, B. (2023). Fuzzy cooperative control for the stabilization of the rotating inverted pendulum system. Journal of Advanced Computational Intelligence and Intelligent Informatics, 27(3), 360–371
6. Schwartz, M., & Manickum, O. (2015). Programming arduino with Labview: Build Interactive and fun learning projects with Arduino using Labview. Packt Pub.
7. Pololu Micro Metal gearmotors. (n.d.). https://www.pololu.com/file/0J1487/pololu-micro-metal-gearmotors_rev-5-1.pdf
8. L298n dual H-Bridge Motor Driver - Handson Tec. (n.d.-a). https://www.handsontec.com/dataspecs/module/L298N%20Motor%20Driver.pdf